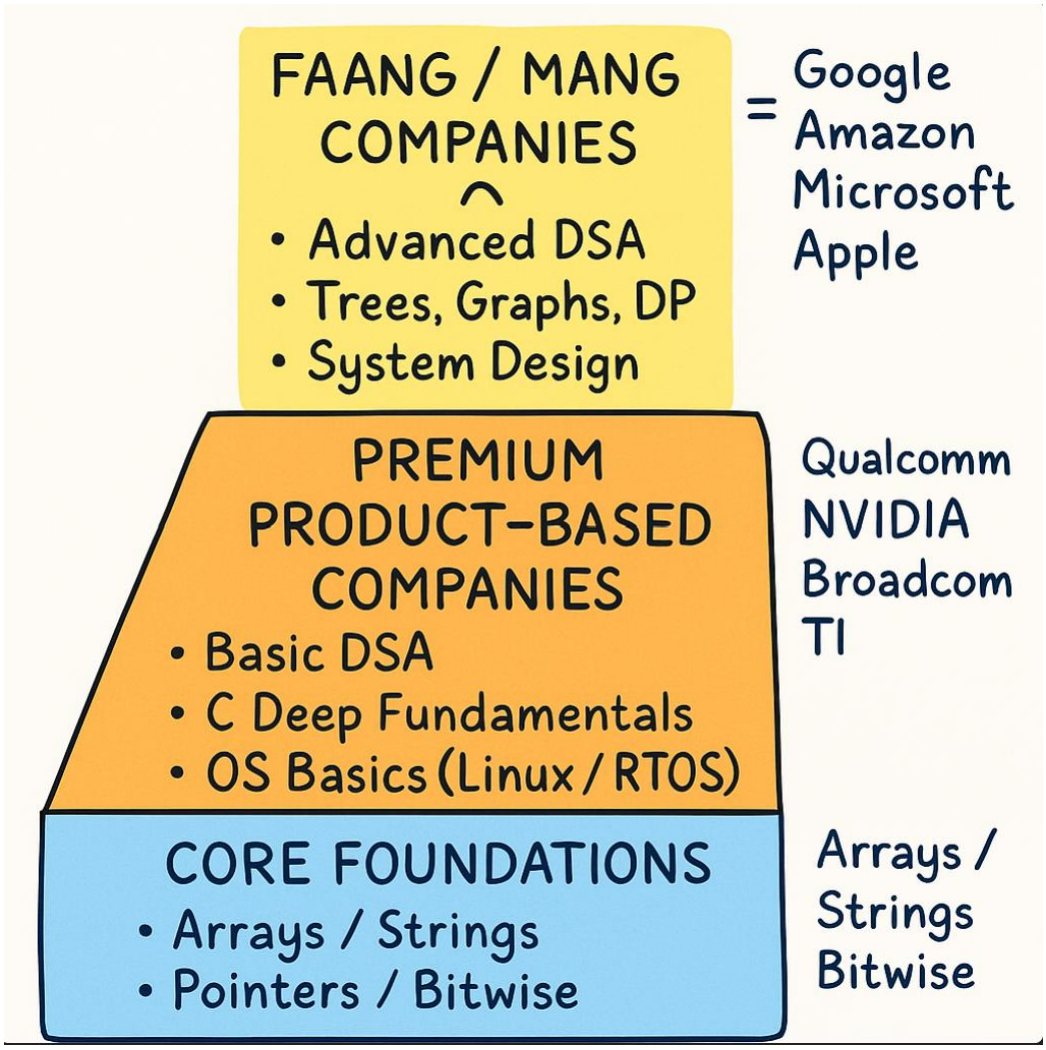



---

📘 **EMBEDDED SYSTEM INTERVIEW PREPARATION — PART 1**

***(For Premium Product-Based Companies — Not MAANG/FANG)***

---

⭐ **Introduction**

If you're preparing for **premium product-based embedded system companies** (e.g., high-quality startups, consumer electronics firms, automotive product companies, networking companies, semiconductor companies, etc.), your interview preparation should be **focused, practical, and core-skill oriented**.

These companies do **NOT** expect MAANG-level DSA.
They want strong fundamentals in **C programming, OS concepts, embedded concepts, and debugging skills**.

This document explains **exactly what to study and how to prepare**.

---

## 🚀 PART A — Core Areas You MUST Master

Premium product companies mainly evaluate four things:

1. **Strong Command Over C Programming**
2. **Solid Understanding of OS Fundamentals (Linux OR RTOS)**
3. **Basic–Intermediate Data Structures (But NOT Leetcode Hard)**
4. **Embedded Concepts + Debugging Mindset**

Let's go step-by-step.

---

### 🔹 1. C Programming Fundamentals (Most Important)

C is the heart of embedded interviews.
You don't need competitive programming.
You need **system-level, memory-level C**.

#### ✔ Topics to Master

- Pointer basics
- Pointer-to-pointer
- Pointer arithmetic
- Arrays & strings
- Function pointers
- Const, volatile, restrict
- Structures, unions, bitfields
- Dynamic memory (malloc/free) — why avoided in embedded
- Memory layout (stack, heap, BSS, data, text)
- Storage classes (auto/static/register/extern)
- Compilation steps, header vs source
- C program execution flow
- Integer promotions & operator precedence

- Common pitfalls and undefined behavior

✔ **What to practice?**

- Reverse a string
- Find length of string
- Print linked list
- Stack/Queue using array
- Remove duplicates from an array
- Bit manipulation problems
- Write your own strcpy, strlen, memcpy, strcat
- Endianness check
- Swap using XOR
- Implement circular buffer
- Simple state machines

This is **enough** for 90% of product-based embedded interviews.

---

🔹 **2. OS Basics (Linux or RTOS)**

Companies expect you to know **how the system works**, NOT deep kernel internals.

✔ **If preparing with Linux:**

- What is a process?
- What is a thread?
- Scheduling basics
- Context switching
- Virtual memory basics
- System calls
- Fork/exec concept
- Mutex vs semaphore
- Deadlock basics
- Interrupt/User space difference

- How device drivers abstract hardware
- File descriptors
- Blocking vs non-blocking
- Poll/select basics
- Simple shell commands (ls, ps, top, dmesg)

✔ **If preparing with RTOS:**

- Task creation
- Task switching
- Task states
- Mutex, semaphore, event groups
- Priority & priority inversion
- Tick interrupt
- Delay API (vTaskDelay / osDelay)
- Queues and message passing
- ISR vs task context
- Memory pools/static allocation

You don't need to read Linux kernel book.
Just **practical OS knowledge**.

---

🔹 **3. Basic Data Structures (NOT Leetcode DSA)**

Premium product companies do **NOT** expect red–black trees or DP questions.
They focus ONLY on structures used in real embedded code.

✔ **Must-know data structures:**

- **Array**
- **String**
- **Linked List (basic)**
- **Stack (array-based)**
- **Queue (array-based / circular buffer)**

**Why?**

Because these are actually used in embedded:

- Linked lists for RTOS queues
- Circular queues for UART buffers
- Stacks for expression parsing & task stacks

✔ **Practice questions:**

- Implement queue using array
- Implement stack
- Insert/delete in linked list
- Reverse a linked list
- Produce-consume using circular buffer
- Basic string handling (uppercase, removal, palindrome)

That's all. Nothing beyond this is required for premium product companies.

---

🔹 **4. Embedded Systems Fundamentals**

You must know how hardware interacts with software.

✔ **Topics:**

- GPIO, UART, I2C, SPI basics
- Registers & bit manipulation
- Polling vs interrupt
- Debouncing
- Timers & counters
- Watchdog
- PWM basics
- Memory-mapped I/O
- Basic board bring-up flow
- Reading a datasheet / reference manual basics
- Booting process overview

- Interrupt flow (ISR, vector table)

✔ **Debugging Mindset:**

- How to trace a bug
- Understanding stack overflow
- Infinite loops
- Race conditions
- How to use logs
- How to reproduce issues
- Boundary cases
- Timing issues
- Handling NULL pointers & corrupted memory

This is what premium companies value most.

---

🚀 **PART B — Interview Strategy & Preparation Plan**

**Step 1: Master C fundamentals (20–30 days)**

Practice small programs every day.

**Step 2: Learn OS basics (Linux OR RTOS) (15 days)**

Just basics + practical understanding.

**Step 3: Solve basic data structure problems (20 days)**

Arrays, strings, linked lists, stack, queue.

**Step 4: Learn core embedded concepts (20 days)**

Timers, interrupts, protocols, state machines.

**Step 5: Build 1–2 hands-on mini projects**

Example:

- I2C temperature sensor
- UART communication
- LED + timer + interrupt demo
- Small FreeRTOS program

**Step 6: Prepare for interview patterns**

- Round 1: C programming
- Round 2: Embedded + OS
- Round 3: Embedded debugging
- Round 4: System thinking scenario
- Round 5: Manager/HR

---

### 🚀 PART C — What They Do NOT Expect

These companies **do NOT** require:

❌ Dynamic programming
❌ Graph algorithms
❌ Trees, heaps, tries
❌ Leetcode medium/hard
❌ Compilers, networking stacks
❌ Deep Linux kernel internals

Your preparation should be **focused**, not scattered.

---

### ⭐ FINAL MESSAGE

To clear premium product-based embedded interviews:

**Master C + OS basics + simple DSA + embedded fundamentals.**
No need for MAANG-style DSA.
Practical knowledge > fancy algorithms.

Stay Tuned for Part 2

Prepared By : EmbeddedShiksha (www.embeddedshiksha.com)